Journal of Machine Learning Research 1 (2026) 1-XX Submitted 1/26; Published 7/26

# kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

**Moussa Kassim**[1,2], **Martin Ocharo**[1,2], **Alessia Vignoli**[3,4]
**Leonardo Tenori**[3,4] and **Stefano Cacciatore**[1,2]
[1] *Bioinformatics Unit, International Center for Genetic Engineering and Biotechnology, Cape Town 7925, South Africa.*
[2] *Department of Integrative Biomedical Sciences,*
*Institute of Infectious Disease & Molecular Medicine (IDM),*
*University of Cape Town, Cape Town 7925, South Africa.*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy.*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy.*
Corresponding author: Stefano Cacciatore, stefano.cacciatore@icgeb.org

**Editor:** To be assigned

## Abstract

KODAMA searches for latent structure by maximizing the cross-validated predictability of an evolving label vector. We present kodama-cpp, a standalone C++17 implementation that preserves this objective while reformulating its numerical execution for multicore CPU, NVIDIA CUDA, and Apple Metal systems. The novelty is not a replacement objective: it is an accelerator-native software design in which one typed core owns float32 data, fold assignments, evolving labels, classifier workspaces, and graph outputs instead of repeatedly crossing an R-package boundary. The CUDA backend provides package-owned exact and recall-tuned IVF-Flat nearest-neighbor search, GPU k-means, label-aware SIMPLS cross-products, reusable device workspaces, and latent-space LDA. The Metal backend provides native exact and IVF-Flat search, GPU k-means, and MPS-assisted SIMPLS projection while retaining the same KNN and PLS-LDA contracts. KODAMA runs remain independent, requested feasible PLS component counts are evaluated directly, and unavailable accelerators raise errors rather than silently executing a CPU substitute. The resulting library exposes KNN and PLS-LDA cross-validation, label optimization, KODAMA matrix construction from data or supplied neighbor graphs, float32 randomized PCA, and visualization primitives through wrapper-independent APIs. We validate mathematical parity, backend identity, runtime, memory behavior, and external-label diagnostics separately so that acceleration claims are not conflated with the internal cross-validation objective.

**Keywords:** KODAMA, unsupervised learning, cross-validation, heterogeneous computing, CUDA, Metal, C++ library, nearest neighbors, PLS-LDA

## 1 Introduction

High-dimensional data analysis often begins with clustering or dimensionality reduction, but neither operation directly asks whether the discovered groups are predictable from the data. KODAMA takes a different view: a useful labeling of samples should be reproducible by a classifier under cross-validation. The labels are not treated as ground truth; they are optimization variables whose quality is measured by held-out prediction.

The original KODAMA paper introduced this idea as knowledge discovery by accuracy maximization, and the subsequent R package made it available as an unsupervised and semi-supervised feature-extraction tool for noisy high-dimensional data. The Bioinformatics package paper describes the algorithm as repeated random label initialization, iterative maximization of cross-validated accuracy by label swapping, and construction of a dissimilarity matrix from the resulting label vectors. The R documentation exposes the same logic through M independent iterative processes, Tcycle optimization steps, KNN or PLS-DA classifiers, and separate KODAMA.matrix and KODAMA.visualization functions.

The repeated classifier fits inside KODAMA create a systems problem that is not resolved by translating individual R expressions into C++. Neighbor indices, fold matrices, class encodings, latent projections, and evolving labels must be reused across many cross-validation calls without changing the accepted label trajectory. A useful accelerator implementation must therefore preserve the mathematical state machine while changing where that state lives and how it moves.

kodama-cpp addresses this problem with a standalone C++17 core and explicit CPU, CUDA, and Apple Metal backends. It currently supports two classifier families in the KODAMA optimization layer: KNN for local neighborhood consistency and PLS-LDA for low-rank linear discrimination. R and Python wrappers call the same typed API, so fold construction, proposals, acceptance decisions, and result metadata are not reimplemented by each language interface.

The principal software novelty has four parts. First, analysis matrices and repeated workspaces are float32-native. Second, nearest-neighbor search, k-means initialization, randomized PCA, SIMPLS, and LDA have backend-specific implementations rather than hidden CPU fallbacks. Third, KODAMA-specific invariants such as independent M runs, one CV evaluation per proposal cycle, constrained groups, and requested PLS component counts remain shared. Fourth, backend identity, timings, memory, search parameters, and prediction diagnostics are returned as data, making heterogeneous execution testable from thin wrappers.

We deliberately frame this contribution as a software-methods paper, not as a claim that KODAMA replaces clustering, semi-supervised learning, or manifold learning. Cross-validated accuracy is the internal optimization criterion used to construct candidate label vectors. External labels, when available, are used only for diagnostics such as adjusted Rand index, silhouette, local purity, and runtime-quality tradeoffs.

## 2 KODAMA objective

Let X in R^(n x p) denote the input matrix, c in {1,...,K}^n a candidate label vector, F a classifier family, and Pi a fold assignment. For each validation sample i, the classifier is trained on samples not in Pi(i) and predicts c_i from x_i. The empirical objective is the held-out accuracy A(c; X, F, Pi). KODAMA searches for a labeling c* that maximizes this quantity, turning unsupervised structure discovery into a self-guided supervised prediction problem.

Table 1: Main contributions of kodama-cpp.

| Contribution | Role in kodama-cpp |
|---|---|
| Mathematical continuity | Preserves cross-validated label predictability, independent M runs, constrained proposals, and the derived KODAMA graph or dissimilarity. |
| Standalone implementation | Moves data ownership and label evolution into a C++17 library so R and Python wrappers share one implementation. |
| Native CUDA backend | Combines exact/IVF-Flat KNN, k-means, label-aware SIMPLS, latent LDA, and reusable device workspaces without FAISS, cuVS, or RAFT runtime links. |
| Native Metal backend | Implements exact/IVF-Flat KNN and k-means with Metal kernels and accelerates SIMPLS projections with Metal Performance Shaders. |
| KODAMA-specific reuse | Caches fold data, compacts active labels, reuses neighbor structures, and keeps M runs independent without changing the objective. |
| Testable backend contract | Returns typed predictions, labels, graphs, timings, memory, search parameters, and actual backend identity, with no silent fallback. |

One KODAMA run starts from an initial partition. In the historical R interface, this can be one label per sample or a clustering-derived vector W; a fraction of samples can be selected for each run, with 75% as the documented default. The current C++ interface keeps the same idea but exposes starting labels, splitting, landmarks, and fixed or constrained groups as typed inputs. During each cycle, misclassified samples or constrained groups generate candidate label moves. A move is accepted if it improves the objective, or under a temperature rule that allows exploration early in the run.

The M runs are independent. This independence is important both statistically, because it averages over random initialization and proposal order, and computationally, because backend schedulers can process runs without changing the objective. The final KODAMA representation is obtained from the ensemble of optimized label vectors: pairs of samples that repeatedly receive compatible labels are pulled closer in the derived graph or dissimilarity, while unstable pairs are weakened.

$$c^\star = \arg\max_{c} \; A(c; X, F, \Pi), \tag{1}$$

where A is the held-out accuracy of classifier family F under fold assignment Pi.

## 2.1 Label-vector search mechanics

The object optimized by KODAMA is the label vector itself. At any time the algorithm holds a candidate vector y on the landmark samples. A classifier is not used to predict an external truth; instead it is trained to reproduce y under cross-validation. A label vector is therefore good when the labels assigned to held-out samples are predictable from the measured variables. The raw accuracy attached to a candidate, acc(y), is the fraction of landmark samples for which the held-out prediction pred_i(y) equals the candidate label y_i.

The first CV pass is run on the initial label vector. This gives both an initial accuracy and a prediction vector. The prediction vector is then used as a proposal guide. If a sample or constrained group repeatedly receives a different predicted label from the classifier, that predicted label is evidence that the current label vector is not aligned with the structure that the classifier can reproduce. KODAMA uses this information to propose a new label vector rather than drawing completely blind random swaps.

Within a cycle, the algorithm chooses a proposal base. In evolutionary mode this is the current accepted vector; otherwise it is the best vector found so far. It samples a number of movable groups according to the adaptive proposal-size schedule. Early cycles can alter many groups, allowing coarse movement through label space. Later cycles alter fewer groups, turning the search into local refinement. For each sampled group, the replacement label is drawn from the empirical distribution of the previous CV predictions among the non-fixed members of that group.

The class-transition step aggregates the same information at label level. It builds a transition matrix N_ab counting samples whose current label is a and whose held-out prediction is b. Large off-diagonal counts indicate that the classifier consistently maps one candidate class into another. KODAMA may therefore propose merging unstable source labels into predicted destination labels, while rejecting moves that collapse the solution to a single class or do not reduce unstable fragmentation.

The many-to-one transition proposal uses the same matrix N. For a source class a and a destination b, the expected count under independence is n_a n_b / n. A candidate absorption is considered only when N_ab exceeds this expectation and also exceeds the number of samples that remain in class a. Candidate destinations are sampled with probability proportional to their positive surplus, and the selected source classes are mapped to the destination only if the move reduces the number of active classes without collapsing all labels into one class.

For the PLS-LDA path, an additional automatic coarsening proposal may be applied before the CV evaluation. It computes the class entropy H = -sum_k p_k log p_k and the effective number of classes K_eff = exp(H). Fragmentation is measured as max(0, log(K / max(1, K_eff))). Classes with high transition instability and small movable size are proposed for merging into their strongest predicted destination; the merge budget is ceil(T_frag max(0, K - K_eff)), where T_frag = fragmentation / (fragmentation + weighted_transition_entropy + 1). This keeps the proposal rule tied to CV confusion rather than to external labels.

After these proposal moves, the proposed vector is evaluated by exactly one new CV pass. The resulting raw accuracy is stored for the best accepted vector, while the acceptance score may include generic guards against degenerate labelings. With guarded diversity, S starts as A sqrt(1 - sum_k p_k^2), where A is raw CV accuracy and p_k is the class proportion. With automatic coarsening, S is reduced by (1 - A) max(H / log n, fragmentation / (1 + fragmentation)). These score terms steer the search away from trivial or excessively fragmented vectors without changing the underlying CV classifier.

The best vector is updated whenever the proposal score improves. In evolutionary mode, a separate current vector is also maintained. A worse proposal can become the current vector with probability exp((score_new - score_current) / tau_t), where tau_t cools toward zero as cycles progress and is also smaller when current accuracy is already high. This simulated-annealing-style rule permits early exploration but makes late-cycle changes conservative. Repeating this process for M independent runs gives an ensemble of high-accuracy label vectors rather than relying on a single local optimum.

### 2.2 Raw accuracy, acceptance score, and diagnostics

It is useful to distinguish three quantities that appear in the implementation. The first is the raw held-out CV accuracy A(y), which is the classifier reproducibility score reported for a label vector. The second is the proposal acceptance score S(y), which is the stochastic-search objective actually used to choose moves when degeneracy guards are enabled. The third is an external diagnostic, such as ARI or embedding compactness, which is never optimized by KODAMA but is reported when reference labels are available.

This distinction is necessary because a high raw CV accuracy alone can be achieved by a collapsed or overly coarse label vector. Such a vector may be easy for the classifier to reproduce but poor as a representation of latent structure. The guarded diversity factor and class-coarsening penalties are therefore part of the current standard KODAMA search score in kodama-cpp: they change which proposals are accepted, while the reported accuracy remains the raw held-out classifier accuracy.

The same distinction also clarifies how to compare KNN and PLS-LDA. KNN can produce very stable local partitions with high raw accuracy, whereas PLS-LDA can favor labelings that are predictable in a low-dimensional discriminant space. For that reason, the benchmark reports both raw CV accuracy and label-quality diagnostics, rather than treating any single scalar as a complete measure of KODAMA quality.

### 2.3 Reproducible KODAMA.matrix procedure

The public KODAMA.matrix routine is reproducible from the following option definitions and run-level procedure. The description names the C++ options because the R and Python wrappers should expose the same contract.

#### 2.3.1 Run-level optimization rule

1. For each run r = 1,...,M, create a random generator with seed seed + r. Copy the input matrix into float32 working storage. Select up to landmarks representative samples by running k-means on all rows with landmarks centers for 10 iterations, then sampling one representative from each non-empty cluster.

2. Initialize the label vector on the selected landmarks. If starting labels are supplied, subset them to the landmarks and apply constrained-majority relabeling when constraints are present. Otherwise run k-means on the landmark matrix with init_k = max(2, min(splitting, number of landmarks)) for 10 iterations.

3. Build CV folds on the landmark set. With no constrain vector, samples are assigned to folds directly. With a constrain vector, each group is assigned as a whole; stratified folds use the group-majority label and non-stratified folds shuffle groups. KODAMA.matrix currently sets the internal CV folds to non-stratified for both KNN and PLS-LDA core paths.

4. At cycle t, choose the proposal base. In evolutionary-chain mode, the base label vector and prediction vector are the current accepted state; otherwise they are the current best state. These carried predictions drive the label proposal, and the proposed label vector is then evaluated by one CV pass.

Table 2: KODAMA.matrix options that determine the optimization.

| Option | Role in the algorithm |
|---|---|
| M / runs | Number of independent KODAMA runs. KODAMAMatrixOptions::runs defaults to 100. Run r uses seed seed + r and writes one length-n label vector to the result matrix. |
| Tcycle / cycles | Number of proposal/evaluation cycles inside each run. KODAMAMatrixOptions::cycles defaults to 20 for compatibility with the historical interface; the benchmark-quality setting used in this manuscript is 100. CoreOptions accepts the same role when the core optimizer is called directly. |
| landmarks | Maximum number of samples optimized directly in one run. The default is 10000; if the requested value is at least n, the implementation uses ceil(0.75 n), then caps the value to [2, n-1]. |
| splitting | Initial number of label classes for each run. If not supplied, splitting is 100 for n < 40000 and 300 otherwise. The run initializes labels by k-means with min(splitting, number of landmarks) clusters. |
| constrain | Optional group vector. Empty means one movable group per sample. Non-empty constraints keep all members of a group in the same CV fold and make the group the unit of label proposal. |
| fixed | Optional binary vector. Fixed samples are included in CV but are not relabeled by proposal moves. |
| graph_neighbors | Number of neighbors retained in the final sparse graph. The actual k is min(graph_neighbors, landmarks, floor(0.75 n - 1)) after enforcing k >= 1. |

5. Sample proposal groups. Let G be the number of groups. With adaptive proposal size enabled, p_t = (t + 1) / (Tcycle + 1), s_t = p_t^2 (3 - 2 p_t), theta_t = 1 - s_t, and q_max(t) = 1 + floor((G - 1) theta_t). Draw q uniformly from {1,...,q_max(t)} and sample q groups without replacement. Without adaptive proposal size, q is uniform on {1,...,G}.

6. For each sampled group, collect the non-fixed members. Their candidate replacement labels are the previous CV predictions for those members. The replacement label is drawn from the empirical frequency of those predicted labels, and all eligible members of the group are relabeled together.

7. Apply class-level transition proposals. The transition matrix has entries N_ab = #{i: current label a, CV prediction b}. The many-to-one proposal selects targets with positive surplus over the independence expectation n_a n_b / n and relabels one or more unstable source classes into the selected target, while rejecting moves that collapse to one class or fail to reduce the class count. For PLS-LDA, an additional coarsening move may merge small unstable classes according to the same transition matrix.

8. Evaluate the proposed labels with exactly one CV pass. Let A_t be the fraction of landmark samples whose proposed label equals the held-out CV prediction. The implementation records the raw accuracy attached to the selected best vector, while the default KODAMA.matrix acceptance score guards against degenerate labelings by multiplying A_t by sqrt(1 - sum_k p_k^2), where p_k is the current class proportion.

When class coarsening is enabled, an additional parsimony term based on label entropy and effective class count is subtracted.

9. Update the best vector whenever the proposal score improves. In evolutionary-chain mode, also update the current vector if the score improves the current score, or with probability exp((score - current_score) / tau_t), where tau_t = max(1e-9, 0.10 max(0, 1 - current_accuracy) (1 - (t + 1) / Tcycle)).

10. After Tcycle cycles, project the optimized landmark labels back to all samples. For the KNN path, each non-landmark receives the majority label among its labeled landmark neighbors in the global graph. For the PLS-LDA path, the final PLS-LDA model predicts the non-landmarks. If constraints are present, the final labels are replaced by the majority label inside each constraint group.

#### 2.3.2 Final graph and dissimilarity

The C++ implementation stores the final KODAMA representation as a sparse neighbor graph rather than materializing a dense n by n matrix by default. First, it constructs a global KNN graph on the original samples using the selected metric and graph_neighbors. This unmodified graph is returned as base_knn.

Let c_i^(r) be the final label of sample i from run r. For each retained edge (i,j) with original distance d_ij, compute V_ij as the number of runs where both endpoint labels are nonzero, and S_ij as the number of those valid runs where the two labels agree. If V_ij = 0 or S_ij = 0, the corrected edge distance is set to infinity. Otherwise a_ij = S_ij / V_ij and d'_ij = (1 + d_ij) / a_ij^2. Each neighbor row is then sorted by d'_ij. The resulting graph is the KODAMA-corrected graph used by KODAMA.visualization.

A dense dissimilarity can be derived from the same agreement statistic by evaluating the formula for all sample pairs. The library keeps the sparse graph form because downstream UMAP, openTSNE, and graph clustering only need local neighborhoods, and the sparse form is the scalable object shared by the available backends.

## 3 Implementation architecture

The core library exposes C++ functions rather than wrapper-specific entry points. A MatrixView accepts double or float inputs, but all analysis paths convert to and retain float32 matrices and workspaces. The core owns fold assignments, compact class encodings, current and best label vectors, classifier buffers, and typed result metadata. The public API is grouped into cross-validation kernels, core label optimization, KODAMA matrix construction, PCA, graph construction, and embedding utilities.

Backend selection is strict and observable. CPU, CUDA, and Metal entry points report the backend that actually executed, and an unavailable accelerator raises an error. This rule is essential for reproducible benchmarking because a nominal GPU call that silently falls back to CPU would preserve outputs while invalidating performance claims.

KNN kernels have backend-specific implementations under one voting contract. The dependency-light CPU path uses a package-owned float32 HNSW implementation. CUDA provides package-owned exact and recall-tuned IVF-Flat search plus GPU k-means using CUDA Toolkit libraries. Apple Metal provides native exact and recall-tuned IVF-Flat search plus GPU k-means using Metal compute kernels. Neighbor structures are reused where the mathematics permits, avoiding repeated wrapper-level index construction.

PLS-LDA uses a SIMPLS strategy following the fastPLS implementation rather than a simplified SVD approximation. CPU, CUDA, and Metal paths operate on float32 matrices and compute class-label cross-products directly instead of constructing dense one-hot response matrices where possible. The requested component count is treated as the evaluated component count whenever mathematically feasible, rather than being replaced by an internal model-selection shortcut. The library exposes no PLS-cKNN classifier; KODAMA optimization is limited to KNN and PLS-LDA.

The library also exposes float32 randomized PCA as a preprocessing and embedding-initialization primitive. A Gaussian range sketch, backend-specific automatic oversampling and power-iteration policy, orthonormal basis, and compact eigensolve produce scores, loadings, singular values, and explained variance. CPU, CUDA, and Metal entry points share this contract. The implementation follows the current fastEmbedR/fastPLS strategy but is standalone and does not link either package, Armadillo, Eigen, or RAFT.

UMAP and openTSNE are direct standalone ports of the pinned current fastEmbedR implementation. UMAP builds binary or fuzzy graphs directly in float32 CSR form, including the smooth-kNN bandwidth calculation and epochs-per-sample schedule; openTSNE uses the matching sparse affinities and exact or FFT-grid repulsion. The CUDA path reuses pooled workspaces, and neither path links fastEmbedR or R.

For graph inputs, where neighbor indices and distances are supplied by the caller, KODAMA keeps the KNN optimizer directly on the supplied graph and avoids rebuilding the neighbor search. If the PLS-LDA optimizer is requested without the original feature matrix, the graph is converted into a PLS-compatible float32 representation with the standard self-tuning normalized Laplacian transform: local row-wise distance scales define edge weights, the graph is symmetrized and degree-normalized, and randomized power iterations produce features for the same PLS-LDA core. This is the only supported graph-to-feature path in the public API.

The graph-only PLS-LDA transform is deterministic conditional on the supplied seed. For row i, sigma_i is the median finite neighbor distance, falling back to the row mean and then to 1e-6 when needed. Each directed edge has weight w_ij = exp(-d_ij^2 / max(1e-12, sigma_i sigma_j)); reciprocal edges are added by taking the larger weight, and the operator is symmetrically normalized as D^{-1/2} W D^{-1/2}. Random Gaussian feature columns are repeatedly multiplied by this sparse operator, orthonormalized for at least eight iterations, and finally standardized before entering the ordinary PLS-LDA core.

The implementation deliberately keeps classifier paths clean. KODAMA optimization is exposed as KNN or PLS-LDA; auxiliary cross-validation kernels exist for testing and benchmarking, but the KODAMA optimizer is not a collection of benchmark-specific branches. The graph, embedding, and clustering utilities are similarly separated from the label-evolution loop.

### 3.1 Implementation novelty

kodama-cpp is an algorithm-preserving systems redesign rather than a new KODAMA objective. The original R implementation remains the reference for accuracy maximization

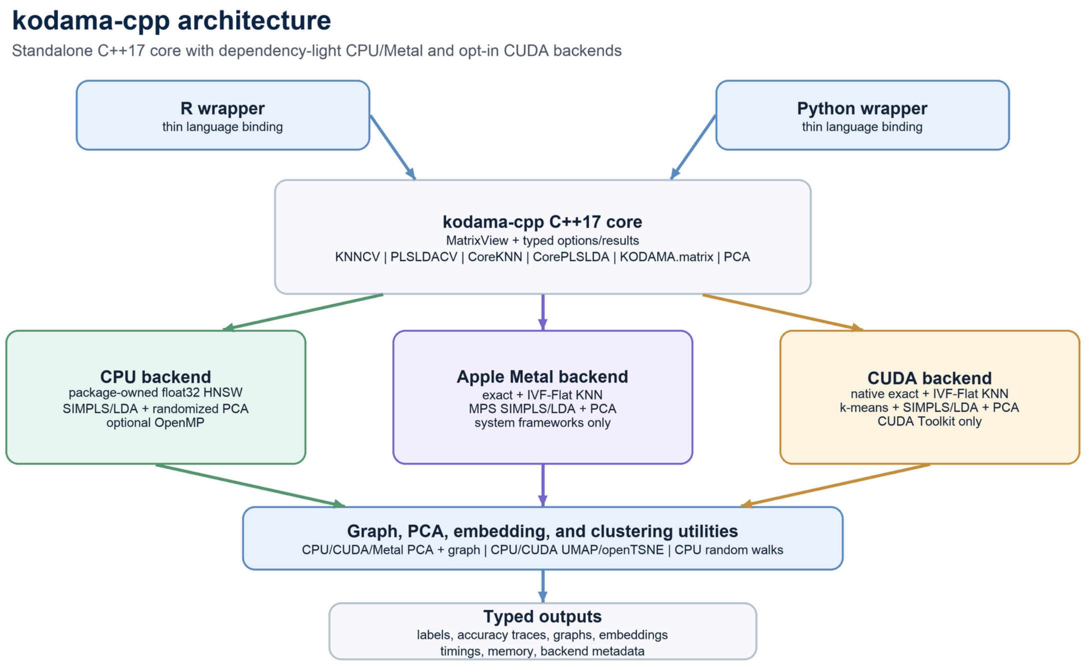


Figure 1: Architecture of kodama-cpp. The C++17 core owns the numerical kernels, CPU, CUDA, and Apple Metal backends, graph utilities, and typed outputs; R and Python remain thin wrappers.

and the ensemble construction. The new contribution is that numerical state previously recreated across wrapper calls is represented once in typed C++ objects and reused across folds, proposal cycles, and independent runs.

This distinction matters because the runtime of KODAMA is the product of several nested operations: M independent searches, Tcycle proposal steps, cross-validation folds, and a classifier fit or vote inside each fold. Accelerating only one matrix multiplication leaves allocation, format conversion, index construction, and host-device traffic in the dominant loop. The implementation therefore optimizes the lifecycle of the complete CV state while retaining one objective evaluation per proposal.

The backends share mathematical invariants but not source-level execution strategies. KNN uses the same metric and voting rules with different search engines. PLS-LDA uses the same SIMPLS deflation and latent LDA model while each accelerator maps the dominant operations to its own memory and matrix-compute model. This separation avoids benchmark-specific branches inside the classifier and makes CPU/accelerator parity an explicit test target.

### 3.2 CUDA backend

The CUDA path is package-owned and uses only CUDA Toolkit libraries. Input matrices are materialized as contiguous float32 buffers. Exact KNN evaluates the requested metric directly, whereas IVF-Flat builds GPU k-means centroids and inverted lists and selects nprobe from an exact pilot query when recall tuning is enabled. The selected nlist, nprobe, pilot recall, and device id are returned with the result. Exact Lloyd assignment uses a bounded 256 MiB score workspace per independent M lane and batches rows when necessary; every row still evaluates every centroid with the same distance and tie rules.

For PLS-LDA, class labels are encoded compactly and the centered X'Y cross-product is formed from class sums on the GPU, avoiding a dense n-by-K one-hot response. Float32 SIMPLS uses reusable thread-local device workspaces that grow to the required capacity. Separate CUDA kernels compute latent projections, class sums, covariance terms, and LDA scores, while fold caches retain train/validation layouts and reusable Gram information.

The CUDA backend reduces allocation and transfer costs but the present release does not claim that the complete M-run ensemble is permanently device resident. Independent runs are scheduled without changing their seeds or proposal trajectories, and small control decisions remain host orchestrated. Fully batched resident M execution is therefore treated as future engineering work, not as evidence used in the current speedup claims.

### 3.3 Apple Metal backend

The Apple backend is implemented in Objective-C++ on Metal rather than emulating CUDA. A persistent Metal state owns the device, command queue, and compute pipelines. Native kernels implement exact KNN, projected IVF-Flat search, Lloyd k-means assignment and centroid updates, and the buffer operations needed by KODAMA initialization and graph construction.

The Metal PLS-LDA path uses float32 class-label cross-products and the same SIMPLS component limit as CPU and CUDA. Metal Performance Shaders execute the dominant Xw and X't matrix products through MPSMatrixMultiplication. The comparatively small response-space power iterations and SIMPLS deflation updates remain host operations. This division is stated explicitly because a native Metal backend does not imply that every scalar operation is a GPU kernel.

Metal is a first-class backend contract: it has named KNNCV, PLSLDACV, CoreKNN, CorePLSLDA, and KODAMAMatrix entry points, reports Metal in result metadata, and is covered by dedicated parity tests. Unsupported Metal operations fail explicitly instead of being relabeled CPU execution. This makes Apple GPU results auditable in the same manner as CUDA results.

Table 3: Backend-specific execution under a shared KODAMA contract.

| Invariant | CUDA realization | Metal realization |
|---|---|---|
| Internal precision | Contiguous float32 host/device matrices and reusable CUDA buffers. | Float32 shared MTLBuffers and MPSMatrix objects. |
| KNN | Exact or recall-tuned IVF-Flat with GPU k-means and returned search metadata. | Exact or recall-tuned IVF-Flat with native Metal k-means. |
| PLS-LDA | GPU label cross-products, SIMPLS workspace, projection, class summaries, and LDA scoring. | Label-aware cross-product with MPS-accelerated X projections and shared SIMPLS/LDA semantics. |
| KODAMA evolution | Same folds, constrained proposals, acceptance score, requested feasible components, and independent M runs. | Same folds, constrained proposals, acceptance score, requested feasible components, and independent M runs. |
| Failure semantics | Unavailable CUDA entry points throw; no CPU fallback is labeled CUDA. | Unavailable Metal entry points throw; no CPU fallback is labeled Metal. |

### 3.4 Shared accelerator contract

### 3.5 Computational scaling

### 3.6 Graph-input KODAMA validation boundary

## 4 Relationship to public KODAMA literature

The contribution of kodama-cpp is not a new definition of KODAMA. It is a reorganization of the published method into a reusable numerical system. The 2014 PNAS paper is the methodological reference for accuracy maximization. The 2017 Bioinformatics paper is the software reference for the R package, including KNN and PLS-DA classifiers, repeated suboptimal runs, and construction of a dissimilarity matrix from the optimized label vectors. The R documentation is the public reference for defaults such as M = 100, Tcycle = 20, 75% sample selection, and the output fields dissimilarity, accuracy, proximity, classification vectors, scores, entropy, and landmarks.

The main differences are architectural and computational. The R package owns a complete R workflow and uses R-package dependencies for data handling, nearest neighbors, and visualization. kodama-cpp owns the numerical kernels and exposes them through C++ types so that R and Python wrappers can share one backend. This makes the comparison with the previous version a compatibility question: held-out label predictability remains the central signal and the output roles can be mapped, while exact trajectories need not match because the current standard adds grouped adaptive proposals, transition-driven coarsening, and a disclosed degeneracy guard. Float32 storage, package-owned CPU/CUDA/Metal search, accelerator kernels, and wrapper-independent graph objects are additional implementation changes.

Table 4: Public API groups in kodama-cpp.

| Layer | Public API | Purpose |
|---|---|---|
| Cross-validation | KNNCV, PLSLDACV | Fold assignment, prediction, fold accuracies, confusion matrices, timing, memory, and backend metadata. |
| Core optimization | CoreKNN, CorePLSLDA | Independent label-evolution runs that maximize KNN or PLS-LDA cross-validated accuracy. |
| KODAMA matrix | KODAMA.matrix / KODAMAMatrix | Landmark selection, splitting, M runs, optimized label vectors, KODAMA dissimilarity, and graph output. |
| Graph-input KODAMA | KODAMA.matrix.graph / KODAMAMatrixFromGraph | Runs the same KNN or PLS-LDA KODAMA optimizer from supplied neighbor indices/distances; KNN reuses the graph directly and PLS-LDA uses self-tuning Laplacian features when the data matrix is unavailable. |
| Principal components | PCA / KODAMA.pca | Float32 randomized PCA scores, loadings, singular values, explained variance, preprocessing vectors, backend identity, and runtime on CPU, CUDA, or Metal. |
| Visualization | KODAMA.visualization | Current fastEmbedR UMAP and openTSNE ports from standard or KODAMA-corrected neighbor graphs; direct float32 CSR construction and binary/fuzzy UMAP graph modes are explicit. |
| Graph and clustering | makeSNNGraph equivalent and random-walk clustering | CPU, CUDA, or Metal neighbor-graph construction followed by an explicit CPU random-walk clustering kernel. |

### 4.1 Implementation novelty relative to the R package

### 4.2 Relationship to semi-supervised learning and cluster validation

KODAMA is related to, but distinct from, semi-supervised learning and weak supervision. Classical semi-supervised graph methods assume that at least some labels are observed and then propagate them under smoothness assumptions, as in Gaussian-field label propagation and local-global consistency. Pseudo-labeling and weak-supervision systems also create or aggregate imperfect labels, but they typically use those labels to train a downstream predictor. KODAMA reverses the emphasis: the label vector itself is the optimization object, and its quality is the ability of a chosen classifier to reproduce it under held-out folds.

The closest evaluation literature is cluster validation by stability or prediction. Methods such as stability-based clustering and prediction strength ask whether a grouping is reproducible under perturbation or held-out prediction. KODAMA uses the same general principle of reproducibility, but internalizes it as the search objective over label vectors. This is why the benchmark reports the internal CV score together with adjusted Rand

Table 5: Dominant computational work in the KODAMA pipeline.

| Component | Dominant work | Implementation implication |
|---|---|---|
| KODAMA runs | M independent runs, each with Tcycle proposal cycles plus one initial CV pass. | Embarrassingly parallel over M; each cycle performs exactly one CV evaluation. |
| KNN core | Neighbor search/build plus O(M Tcycle n_L k) voting on landmarks, where n_L is the landmark count and k is the KNN predictor size. | Graph-input KNN removes the search/build term and reuses supplied indices and distances. |
| PLS-LDA core | O(M Tcycle folds C_PLS(n_train, p, h)) for h requested components, plus latent-space LDA scoring. | Float32 label-aware SIMPLS and fold buffers reduce constants but do not change the repeated-CV structure. |
| Graph correction | O(M E) agreement counting for E retained graph edges, followed by row-wise neighbor sorting. | Sparse by default; dense n by n dissimilarities are avoided for scalability. |
| Visualization | Runs on the base or KODAMA-corrected sparse graph using UMAP/openTSNE helpers. | Visualization time is reported separately because it is not part of the CV optimization objective. |

Table 6: Interpretation of graph-input KODAMA paths.

| Path | Implementation | Interpretation |
|---|---|---|
| KNN graph input | Uses supplied neighbor indices/distances directly in CoreKNNGraph and KODAMAMatrixFromGraph. | Accepted as the graph-input path because it preserves the KNN voting mathematics and removes repeated search work. |
| PLS-LDA graph input with X | When the original data matrix is supplied with the graph, PLS-LDA uses float32 X and the graph supplies initialization/projection support. | Equivalent classifier path to data-input PLS-LDA; graph handling is an input-format extension. |
| PLS-LDA graph-only input | When X is unavailable, the graph is transformed into self-tuning normalized Laplacian features before CorePLSLDA. | Exposed as an optional fallback, not as evidence that graph-only PLS-LDA matches data-input PLS-LDA in all datasets. |
| Validation boundary | Wrapper tests exercise both KNN and PLS-LDA graph-input calls; benchmark comparisons must use the same M, Tcycle, seed, and supplied graph. | The manuscript does not use graph-only PLS-LDA results to support data-input KODAMA performance claims. |

index, silhouette, local purity, and active class count rather than presenting CV accuracy alone as proof of external correctness.

The manuscript also follows warnings from the model-selection and circular-analysis literature. Cross-validation can be biased when the same data are used both to choose a model and to estimate final performance; feature selection outside the resampling loop is a classic example. In kodama-cpp, the internal CV score is intentionally the search criterion,

Table 7: Compatibility with the public KODAMA R interface and literature.

| Aspect | Public R/literature contract | kodama-cpp |
|---|---|---|
| Objective | KODAMA maximizes cross-validated accuracy of an evolving label vector. | Held-out label predictability remains the classifier signal and raw accuracy is reported; the standard search ranks proposals with a disclosed label-only degeneracy guard. |
| Search dynamics | The historical core evolves labels through its published stochastic accuracy-maximization procedure. | Grouped adaptive moves and transition-driven coarsening are explicit search extensions; they use CV predictions but never reference labels. |
| Independent runs | R documentation reports M iterative processes and Tcycle optimization cycles. | KODAMAMatrixOptions exposes runs and cycles; runs remain independent across CPU, CUDA, and Metal backends. |
| Sample selection | The documented FUN_SAM default selects 75% of samples per iterative process. | Landmark and splitting controls preserve the same sampling role while exposing it in typed options. |
| Classifiers | The R package documents KNN and PLS-DA classifiers. | The KODAMA optimizer exposes KNN and SIMPLS + PLS-LDA; component count is explicit. |
| Output contract | The R interface returns dissimilarity, accuracy, proximity, label vectors, scores, entropy, and landmarks. | The C++ result returns optimized labels, accuracies, graphs, timings, memory, and backend metadata for wrappers. |
| Architecture | The R package is built around R workflows and R dependencies. | The core is R/Python independent, CMake-installable, and accelerated where available. |

so external claims must come from separate diagnostics, transparent parameter sensitivity, and comparisons with standard UMAP and t-SNE on the same datasets.

## 5 Evaluation

We evaluate kodama-cpp at three levels. First, kernel benchmarks isolate KNNCV and PLSLDACV from the full KODAMA matrix pipeline so that nearest-neighbor search and PLS-LDA fitting can be interpreted separately. Second, core optimizer benchmarks measure repeated label-vector evolution over multiple seeds. Third, matrix-level validation measures end-to-end KODAMA.matrix calls and reports both cross-validated accuracy and external-label diagnostics where reference labels are available.

The evaluation is intentionally separated into these layers because KODAMA has several scaling regimes. KNN acceleration is dominated by neighbor search and voting. PLS-LDA acceleration is dominated by repeated SIMPLS and LDA work across folds and cycles. The final graph and visualization stages depend primarily on the sparse graph size. Reporting the

Table 8: Implementation innovations in kodama-cpp.

| Innovation | What changes in kodama-cpp |
|---|---|
| State ownership | Moves folds, labels, classifier workspaces, and graph outputs from repeated R-level calls into reusable C++ objects. |
| Backend-native classifiers | Keeps KNN and PLS-LDA mathematically shared but implements their dominant operations separately for CPU, CUDA, and Metal. |
| Float32 execution | Uses float32 analysis matrices and accelerator workspaces end to end while accepting wrapper-provided double inputs at the boundary. |
| Label-aware SIMPLS | Computes PLS-LDA response cross-products from compact labels instead of dense one-hot matrices and evaluates the requested feasible component count. |
| Accelerated search | Provides package-owned CPU HNSW, CUDA exact/recall-tuned IVF-Flat search, and Metal exact/recall-tuned IVF-Flat search. |
| Reproducible heterogeneity | Keeps M runs independent and reports actual backend and search metadata so acceleration can be tested without altering the objective. |
| Search evolution | Makes grouped adaptive proposals, transition-driven class moves, and the label-only degeneracy guard explicit and separately reports their raw CV accuracy and acceptance score. |

Table 9: Literature positioning of KODAMA.

| Literature area | Typical goal | Role of KODAMA |
|---|---|---|
| Semi-supervised graph learning | Label propagation and local-global consistency start from observed labels and smooth them on a graph. | KODAMA starts from synthetic or supplied candidate labels and optimizes their cross-validated predictability. |
| Pseudo-labeling and weak supervision | Pseudo-labeling and data programming create imperfect labels to train predictive models. | KODAMA treats the label vector as the object being discovered; the classifier is the measuring instrument. |
| Cluster stability and prediction strength | Stability methods evaluate whether clusters survive perturbation or prediction on held-out samples. | KODAMA turns held-out predictability into an optimization objective, then reports external diagnostics separately. |
| Manifold visualization | UMAP and t-SNE optimize low-dimensional layouts from neighbor or affinity structures. | KODAMA supplies a corrected graph to the same visualization step; it does not replace those embedding objectives. |
| Open-source ML software | JMLR software papers emphasize clear APIs, reproducibility, tests, licensing, and benchmark transparency. | kodama-cpp separates the numerical core from R/Python wrappers and records backend parameters and validation evidence. |

layers separately makes it possible to attribute a speedup to the part of the implementation that produced it.

The evaluation also separates optimization from validation. Held-out CV accuracy is the internal classifier signal, while the current standard ranks proposals with the disclosed label-only acceptance score. Neither quantity is interpreted as external biological, chemical, or semantic truth. When reference labels are available, they are held outside the optimizer and reported after the fact. This distinction follows the broader cautionary literature on model selection, feature selection bias, and circular analysis.

All timings in this section are wall-clock seconds from the benchmark drivers, and all accuracy values are computed on held-out cross-validation folds. CPU kernel measurements use the configured single-thread CPU path unless explicitly labeled otherwise; CUDA measurements use the same CUDA Toolkit runtime used by the test suite. Tables are intentionally separated by scope: kernel-level rows do not claim end-to-end speedups, and graph-input rows are treated as API validation unless paired with data-input KODAMA results under the same M, Tcycle, seed, and graph settings.

### 5.1 Benchmark protocol and data coverage

Table 10: Benchmark protocol for the current evaluation.

| Item | Value |
|---|---|
| Date | 2026-07-19 UTC for current-CRAN and CI/coverage validation; 2026-07-18 UTC for CUDA visualization validation; M/Tcycle validation dated 2026-07-16 UTC; earlier broad kernel snapshot dated 2026-07-06 UTC |
| GPU | NVIDIA GeForce RTX 5060 Ti, 16 GB device memory, driver 595.71.05 |
| Build validation | Fresh CUDA 13.2 build succeeded; CTest passed 4/4 configured tests in 1.91 s |
| Runtime | CUDA Toolkit only for native search/PLS paths; no FAISS, cuVS, RAFT, RMM, or Armadillo link |
| Data format | RData lists exported to contiguous float32 row-major matrices |
| CPU setting | Single-thread CPU kernels unless otherwise stated |
| CV kernels | KNNCV and PLSLDACV measured independently from the full matrix pipeline |
| Core optimizer | Three seeds per dataset for CoreKNN and CorePLSLDA medians |
| Reported metrics | Wall time, peak memory where available, CV accuracy, ARI, and active class count |

### 5.2 Evaluation guardrails and ablations

### 5.3 Dependency-free CPU and Apple Metal validation

The dependency-light backend experiment was run on macOS with CUDA and OpenMP disabled. The CPU build therefore measures the package-owned HNSW and float32 SIMPLS/LDA implementations, while the Metal build links only Apple system frameworks. Exact CPU/Metal equality of the reported classification metrics is the primary acceptance criterion; timings are secondary evidence.

Table 11: Datasets copied into the benchmark data root for the current evaluation.

| Dataset | Samples | Variables | Classes |
|---|---|---|---|
| COIL20 | 1,440 | 16,384 | 20 |
| FashionMNIST | 70,000 | 784 | 10 |
| FlowRepository | 5,220,347 | 32 | 13 |
| MNIST | 70,000 | 784 | 10 |
| Macosko2015_retina | 44,808 | 50 | 12 |
| MetRef | 873 | 375 | 22 |
| TabulaMuris | 70,118 | 50 | 56 |
| USPS | 11,000 | 256 | 10 |
| flow18 | 1,000,021 | 11 | 16 |
| imagenet | 1,281,167 | 1,024 | 1,000 |
| mass41 | 965,282 | 14 | 17 |

Table 12: Evaluation safeguards used to avoid over-interpreting the internal CV objective.

| Concern | Manuscript position | Evidence reported or controlled |
|---|---|---|
| Circularity | CV accuracy is the internal objective, not an external truth score. | Report ARI, silhouette, local purity, active classes, and standard embedding baselines after optimization. |
| Overclaiming | Present kodama-cpp as a KODAMA implementation and acceleration library, not a new universal learning paradigm. | State methodological continuity with the 2014 and 2017 KODAMA publications and limit novelty claims to architecture and numerics. |
| Parameter dependence | M, Tcycle, landmarks, splitting, KNN k, graph k, and PLS components can affect results. | Use predefined sensitivity experiments and report parameter settings in every benchmark table and plot. |
| Visualization bias | A visually appealing layout is not itself evidence of label quality. | Run classic UMAP/openTSNE and KODAMA-corrected UMAP/openTSNE with the same implementation and record compactness metrics. |
| Runtime attribution | End-to-end speedups can hide which kernel produced the benefit. | Separate CV kernels, core optimizer, KODAMA.matrix, graph construction, and visualization timings. |
| Reproducibility | Wrapper behavior must match the standalone core. | Record CMake settings, backend metadata, R CMD check, pytest, CTest, seeds, and benchmark command lines. |

Metal IVF-Flat remains an explicit option rather than an automatic replacement for exact search. On MNIST10k, exact and recall-tuned IVF KNNCV both produced accuracy 0.9490, while elapsed time changed from 2.054 to 1.662 s. A 0.99 pilot-recall target was rejected because accuracy decreased to 0.9482; the accepted automatic pilot target is 0.999.

Table 13: Ablation matrix for the release benchmark.

| Ablation | Comparison | Question answered |
|---|---|---|
| Search evolution | Raw-accuracy proposals versus the standard grouped/guarded search | Separates the current search extension from backend acceleration and from the unchanged CV classifiers. |
| Classifier | KNN versus PLS-LDA | Tests whether local-neighbor and latent-linear predictability give complementary label vectors. |
| Graph correction | Standard graph versus KODAMA-corrected graph | Isolates whether the optimized label ensemble improves downstream UMAP/openTSNE structure. |
| Search depth | Tcycle = 20, 50, 100 | Measures whether additional proposal cycles improve CV accuracy or reduce fragmentation. |
| Ensemble size | M = 20, 50, 100 | Measures whether more independent runs stabilize the final graph and best-run diagnostics. |
| Landmark and splitting controls | Predeclared values, not per-dataset visual tuning | Checks sensitivity of the initial label space and directly optimized sample subset. |
| Backend | Single-core CPU, 4-core MultiCPU, CUDA, and Apple Metal where available | Separates mathematical equivalence from scheduling and hardware acceleration. |
| Wrapper parity | C++ core, R wrapper, Python wrapper | Ensures language bindings are thin and do not introduce independent numerical behavior. |

End-to-end KODAMA can still favor exact search when repeated IVF training costs more than the saved query time.

Table 14: Dependency-free CPU and native Apple Metal validation. Quality entries are CPU/Metal where two values are shown.

| Dataset | Scope | CPU s | Metal s | Speedup | Quality |
|---|---|---|---|---|---|
| MetRef | KNNCV | 11.145 | 0.026 | 425x | accuracy 0.827033 / 0.827033 |
| MetRef | PLSLDACV, 50 components | 3.395 | 1.054 | 3.2x | accuracy 0.991982 / 0.991982 |
| MetRef | KODAMA KNN, M=10 T=20 | 21.709 | 0.171 | 127x | CV 0.978626; ARI 0.093922; 15 classes |
| MetRef | KODAMA PLS-LDA, M=10 T=20 | 178.403 | 78.746 | 2.27x | CV 0.983206; ARI 0.835153; 32 classes |

### 5.4 CUDA workstation validation

The benchmark run used copied float32 RData datasets spanning n = 873 to 5,220,347 samples and p = 11 to 16,384 variables. The CUDA machine first rebuilt the release candidate with CUDA 13.2 and reran CTest; all four configured tests passed. The broad kernel and core tables retain the earlier release-validation snapshot for continuity, whereas the native-CUDA table reports the dependency-distilled search implementation introduced afterward.

The current package-owned CUDA IVF-Flat path was checked on MNIST70k and MetRef with five-fold cosine KNNCV and k = 10. On MNIST70k it produced accuracy 0.973857 in 4.233 s with nlist = 237 and maximum nprobe = 32, compared with 0.973029 in 4.753 s for the recorded former FAISS/cuVS row. On MetRef it produced 0.816724 in 0.195 s, compared with 0.814433 in 0.297 s. These are implementation spot checks rather than repeated-run performance estimates.

The M = 100, Tcycle = 100 MetRef validation shows why cross-validated accuracy is reported together with label-quality diagnostics. Both optimizers reached best CV accuracy 1.000, but KNN selected only three classes and had ARI 0.052. PLS-LDA selected 21 classes against 22 reference classes, attained ARI 0.770, and increased truth-label UMAP silhouette from 0.129 for the classic graph to 0.783 for the KODAMA graph. These reference labels were never supplied to optimization. We therefore report CV accuracy, active classes, external-label agreement when available, and downstream compactness as complementary rather than interchangeable diagnostics.

The dependency-distilled CUDA build links CUDA Toolkit libraries but no FAISS, cuVS, RAFT, or RMM soname. Binary wrappers therefore need to locate the CUDA runtime selected at build time, but do not need to provision the removed search libraries.

Table 15: Dependency-distilled native CUDA KNN spot checks.

| Dataset | Path | Accuracy | Seconds | nlist | max nprobe | Pilot recall |
|---|---|---|---|---|---|---|
| MNIST70k | native IVF-Flat | 0.973857 | 4.233 | 237 | 32 | 0.994531 |
| MetRef | native IVF-Flat | 0.816724 | 0.195 | 27 | 20 | 0.991406 |

#### 5.4.1 Earlier accelerator validation snapshot

Table 16: Earlier CPU/CUDA cross-validation kernel measurements retained for release-history continuity. Accuracy is reported as CPU/CUDA.

| Dataset | Kernel | CPU s | CUDA s | Speedup | Acc CPU/CUDA | Delta | Comp. |
|---|---|---|---|---|---|---|---|
| COIL20 | KNNCV | 5.455 | 1.435 | 3.8x | 0.917/0.916 | -0.000695 | - |
| MNIST | KNNCV | 76.769 | 4.753 | 16.2x | 0.974/0.973 | -0.000928 | - |
| MetRef | KNNCV | 0.163 | 0.297 | 0.6x | 0.821/0.814 | -0.006873 | - |
| MetRef | PLSLDACV | 3.196 | 0.118 | 27.2x | 0.992/0.992 | 0.000000 | 50 |
| USPS | KNNCV | 2.273 | 0.429 | 5.3x | 0.688/0.688 | -0.000819 | - |
| USPS | PLSLDACV | 3.760 | 0.158 | 23.8x | 0.684/0.687 | +0.002727 | 50 |
| mass41 | KNNCV | 366.883 | 3.692 | 99.4x | 0.912/0.912 | -0.000066 | - |

#### 5.4.2 Core optimizer medians

#### 5.4.3 MultiCPU versus GPU KODAMA comparison

The backend comparison is separated by scope. Core-optimizer rows measure the repeated CV label-search kernels without final graph construction, while the KODAMA.matrix row

Table 17: Medians for the CoreKNN and CorePLSLDA optimization paths. Accuracy and ARI are reported as CPU/CUDA.

| Dataset | Core | CPU s | CUDA s | Speedup | Acc | ARI | Classes |
|---|---|---|---|---|---|---|---|
| MetRef | CoreKNN | 0.147 | 0.122 | 1.2x | 0.963/0.947 | 0.271/0.313 | 16/16 |
| MetRef | CorePLSLDA | 16.597 | 0.626 | 26.5x | 0.885/0.838 | 0.357/0.366 | 53/48 |
| USPS | CoreKNN | 2.282 | 0.242 | 9.4x | 0.969/0.968 | 0.216/0.240 | 59/53 |
| USPS | CorePLSLDA | 20.189 | 1.950 | 10.4x | 0.938/0.910 | 0.140/0.146 | 80/77 |

includes landmark selection, label search, projection, and graph correction on a small end-to-end validation dataset. This prevents kernel timings from being reported as full pipeline timings.

The rows below therefore state their scope explicitly. CPU rows use the configured CPU backend, CUDA rows use the CUDA backend, and differences in CPU/CUDA accuracy are treated as reproducibility tolerances caused by stochastic search, float32 arithmetic, and approximate nearest-neighbor behavior rather than as evidence of different mathematical objectives.

Table 18: KODAMA core optimizer CPU/CUDA comparison. Scope is reported explicitly because these rows do not include final graph construction.

| Dataset | Classifier | Backend | Seconds | CV acc | ARI | Classes | Scope |
|---|---|---|---|---|---|---|---|
| MetRef | KNN | CPU | 0.147 | 0.963 | 0.271 | 16 | core optimizer |
| MetRef | KNN | CUDA | 0.122 | 0.947 | 0.313 | 16 | core optimizer |
| MetRef | PLS-LDA | CPU | 16.597 | 0.885 | 0.357 | 53 | core optimizer |
| MetRef | PLS-LDA | CUDA | 0.626 | 0.838 | 0.366 | 48 | core optimizer |
| USPS | KNN | CPU | 2.282 | 0.969 | 0.216 | 59 | core optimizer |
| USPS | KNN | CUDA | 0.242 | 0.968 | 0.240 | 53 | core optimizer |
| USPS | PLS-LDA | CPU | 20.189 | 0.938 | 0.140 | 80 | core optimizer |
| USPS | PLS-LDA | CUDA | 1.950 | 0.910 | 0.146 | 77 | core optimizer |

#### 5.4.4 Legacy KNN-compatible predecessor comparison

A legacy KNN-compatible predecessor experiment used MetRef, KNN, M = 100, Tcycle = 100, 655 effective landmarks under the historical 75% rule, splitting = 50, k = 30, graph k = 100, and seed 1234. Splitting = 50 matches the internal k-means initialization in KODAMA R 2.4.1. This legacy release is used because it exposes the selectable KNN path needed for a KNN-to-KNN comparison; it is not presented as the current CRAN release. PLS is excluded because the historical PLS-DA decoder and current SIMPLS plus latent-space LDA are different classifiers.

The current grouped and guarded proposals extend the historical stochastic search, so this is a matched-settings predecessor and systems comparison rather than label-trajectory parity. Current single-core execution was 1.58 times slower than KODAMA R 2.4.1 on this small case, whereas four-core execution was 2.59 times faster and CUDA was 258.6 times faster. Every row reached best raw CV accuracy 1.000; the different median scores and label diagnostics are reported rather than interpreted as backend equivalence.

Table 19: Legacy KNN-compatible MetRef comparison at M = 100 and Tcycle = 100. Speedup is relative to KODAMA R 2.4.1; this is not the current CRAN release.

| Implementation | Backend | Workers | Seconds | Speedup | CV best/median | ARI selected/median | Classes selected/median |
|---|---|---|---|---|---|---|---|
| KODAMA R 2.4.1 | CPU | 1 | 610.543 | 1.00x | 1.000/0.961 | 0.000/0.054 | 2/6 |
| kodama-cpp | CPU | 1 | 965.348 | 0.63x | 1.000/0.874 | 0.045/0.071 | 2/8.5 |
| kodama-cpp | CPU | 4 | 235.547 | 2.59x | 1.000/0.874 | 0.045/0.071 | 2/8.5 |
| kodama-cpp | CUDA | 4 | 2.361 | 258.6x | 1.000/0.882 | 0.052/0.069 | 3/8.5 |

#### 5.4.5 Current CRAN systems comparison

A separate short-run systems comparison addresses the current public release. MetRef was run with M = 20, Tcycle = 20, requested landmarks = 100000 (655 effective landmarks under the retained 75% rule), splitting = 100, graph k = 500, ncomp = 50, seed 1234, and four CPU workers. The CV-selected run is chosen by raw held-out accuracy; ARI and class count are computed only afterward. KODAMA 3.3 automatically selects its PLS route because its deprecated FUN argument is ignored, so this is not classifier- or trajectory-parity with kodama-cpp SIMPLS plus latent-space LDA.

Current CRAN KODAMA 3.3 completed in 344.914 seconds, compared with 822.989 seconds for kodama-cpp CPU4; thus KODAMA 3.3 was 2.39 times faster in this limited check. It also had higher selected raw CV accuracy and selected ARI. These unfavorable current results are retained and are not combined with the classifier-matched legacy KNN comparison. An exactly matched CUDA row is omitted because graph k = 500 exceeds the native CUDA builder limit of 256; neither landmarks nor graph k was changed silently to create an accelerator row.

Table 20: Short MetRef current-release systems comparison at M = 20 and Tcycle = 20. KODAMA 3.3 and kodama-cpp use different PLS classifier routes; this is not classifier parity.

| Implementation | Classifier route | Backend | Workers | Seconds | CV selected/median | ARI selected/median | Classes selected/median |
|---|---|---|---|---|---|---|---|
| KODAMA 3.3 | automatic PLS | CPU | 4 | 344.914 | 0.9893 / 0.9740 | 0.7004 / 0.6961 | 20 / 20 |
| kodama-cpp 0.1.0 | SIMPLS + LDA | CPU | 4 | 822.989 | 0.9802 / 0.9550 | 0.6430 / 0.6584 | 30 / 35 |

Table 21: KODAMA.matrix CUDA validation on MetRef with M = 100 and Tcycle = 100.

| Classifier | Elapsed s | Runtime s | CV acc | ARI | Med. classes | Class range |
|---|---|---|---|---|---|---|
| KNN | 3.115 | 2.646 | 1.0000/0.8809 | 0.0517/0.0613 | 9 | 2-52 |
| PLS-LDA | 584.089 | 584.074 | 1.0000/0.9924 | 0.7700/0.7048 | 24 | 15-34 |

#### 5.4.6 Classic versus KODAMA visualization validation

Because the final KODAMA object is often interpreted through a two-dimensional layout, we added an explicit visualization comparison against standard UMAP and openTSNE. The driver runs the same embedding implementation on either the ordinary neighbor graph or the KODAMA-corrected graph, then reports truth-label silhouette, local label purity, KODAMA label ARI, active classes, and wall-clock time. This makes the visualization comparison separate from the CV objective and prevents the evaluation from relying on a single favorable small dataset.

The release-validation panel used CUDA with M = 100, Tcycle = 100, landmarks = 100000 subject to the historical 75% rule, KNN predictor k = 30, graph k = 100, embedding k = 30, openTSNE perplexity = 30, and seed 1234. It evaluated MetRef, PBMC3K PCA50, OptDigits, USPS, and Macosko2015 retina with both KNN and PLS-LDA. Reference labels were withheld from optimization and used only for the reported ARI, silhouette, purity, and compactness diagnostics.

The result is deliberately mixed. PLS-LDA increased truth-label silhouette on both embeddings for MetRef (UMAP 0.105 to 0.815; openTSNE 0.067 to 0.530) and PBMC3K PCA50 (0.410 to 0.484; 0.349 to 0.402), but reduced it on OptDigits, USPS, and Macosko2015 retina. KNN reduced silhouette on all five datasets despite best CV accuracy between 0.9927 and 1.0000. Thus a highly predictable optimized label vector is not sufficient evidence of improved external-label recovery or visualization, and no universal visualization-improvement claim is made.

Complete KODAMA.matrix wall time ranged from 2.967 to 97.295 seconds for KNN and from 486.707 to 708.579 seconds for PLS-LDA. The paired embeddings themselves required less than 0.88 seconds each in this panel, so the reported timing distinction correctly attributes the dominant cost to label optimization rather than to UMAP or openTSNE.

Table 22: CUDA M=100, Tcycle=100 comparison of standard UMAP/openTSNE with KODAMA-corrected graphs. Silhouette and purity are computed against external labels used only after optimization; ARI and class count refer to the CV-selected KODAMA label vector.

| Dataset | Classifier | Embedding | Sil. classic → KODAMA | Δ sil. | Purity classic → KODAMA | ARI/classes | KODAMA s |
|---|---|---|---|---|---|---|---|
| MetRef | KNN | UMAP | 0.105 -> 0.016 | -0.090 | 0.590 -> 0.575 | 0.049 / 3 | 3.381 |
| MetRef | KNN | openTSNE | 0.067 -> 0.018 | -0.049 | 0.616 -> 0.608 | 0.049 / 3 | 3.381 |
| MetRef | PLS-LDA | UMAP | 0.105 -> 0.815 | +0.709 | 0.590 -> 0.966 | 0.879 / 28 | 589.633 |
| MetRef | PLS-LDA | openTSNE | 0.067 -> 0.530 | +0.463 | 0.616 -> 0.904 | 0.879 / 28 | 589.633 |
| PBMC3K PCA50 | KNN | UMAP | 0.410 -> 0.387 | -0.023 | 0.830 -> 0.796 | 0.374 / 3 | 2.967 |
| PBMC3K PCA50 | KNN | openTSNE | 0.349 -> 0.336 | -0.013 | 0.826 -> 0.805 | 0.374 / 3 | 2.967 |
| PBMC3K PCA50 | PLS-LDA | UMAP | 0.410 -> 0.484 | +0.075 | 0.830 -> 0.820 | 0.594 / 11 | 486.707 |
| PBMC3K PCA50 | PLS-LDA | openTSNE | 0.349 -> 0.402 | +0.053 | 0.826 -> 0.825 | 0.594 / 11 | 486.707 |
| OptDigits | KNN | UMAP | 0.700 -> 0.600 | -0.100 | 0.974 -> 0.949 | 0.603 / 28 | 3.920 |
| OptDigits | KNN | openTSNE | 0.563 -> 0.446 | -0.117 | 0.972 -> 0.959 | 0.603 / 28 | 3.920 |
| OptDigits | PLS-LDA | UMAP | 0.700 -> 0.548 | -0.153 | 0.974 -> 0.944 | 0.350 / 47 | 496.673 |
| OptDigits | PLS-LDA | openTSNE | 0.563 -> 0.470 | -0.094 | 0.972 -> 0.960 | 0.350 / 47 | 496.673 |
| USPS | KNN | UMAP | 0.153 -> 0.071 | -0.081 | 0.698 -> 0.651 | 0.300 / 34 | 19.141 |
| USPS | KNN | openTSNE | 0.212 -> 0.090 | -0.121 | 0.711 -> 0.672 | 0.300 / 34 | 19.141 |
| USPS | PLS-LDA | UMAP | 0.153 -> 0.082 | -0.071 | 0.698 -> 0.598 | 0.109 / 70 | 663.782 |
| USPS | PLS-LDA | openTSNE | 0.212 -> 0.081 | -0.131 | 0.711 -> 0.643 | 0.109 / 70 | 663.782 |
| Macosko retina | KNN | UMAP | 0.469 -> 0.183 | -0.286 | 0.965 -> 0.953 | 0.171 / 73 | 97.295 |
| Macosko retina | KNN | openTSNE | 0.171 -> -0.022 | -0.193 | 0.962 -> 0.959 | 0.171 / 73 | 97.295 |
| Macosko retina | PLS-LDA | UMAP | 0.469 -> 0.176 | -0.293 | 0.965 -> 0.953 | 0.410 / 30 | 708.579 |
| Macosko retina | PLS-LDA | openTSNE | 0.171 -> 0.060 | -0.111 | 0.962 -> 0.958 | 0.410 / 30 | 708.579 |

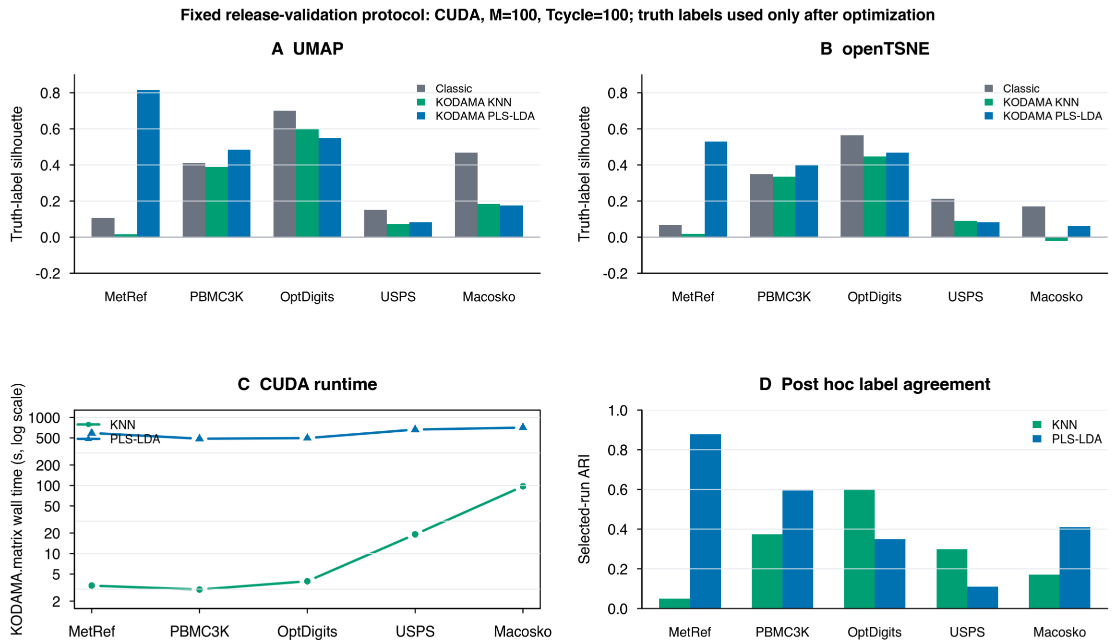


Figure 2: Fixed M = 100 and Tcycle = 100 nonspatial visualization validation. Panels A and B compare truth-label silhouette; panel C reports complete KODAMA.matrix wall time on a logarithmic scale; panel D reports selected-run ARI. Reference labels were used only after optimization.

### 5.5 Sensitivity of M and Tcycle

Because KODAMA has two nested stochastic controls, M and Tcycle cannot be justified by visual quality alone. Tcycle controls the depth of label-vector evolution inside one run, whereas M controls how many independent local optima enter the final ensemble. The named release-validation experiment uses MetRef and USPS with both KODAMA classifiers: fixing M = 100 while varying Tcycle in {20, 50, 100}, and fixing Tcycle = 100 while varying M in {20, 50, 100}. It uses landmarks = 100000 subject to the historical 75% rule when n is no larger than landmarks, the default splitting rule, ncomp = 50, knn.k = 30, and a recorded seed.

The Tcycle sweep reports best and median cross-validated accuracy, adjusted Rand index against reference labels used only after optimization, active-class count, and runtime. On USPS, increasing Tcycle from 20 to 100 raised best/median CV accuracy from 0.9662/0.9414 to 0.9997/0.9925 for KNN and from 0.9383/0.9220 to 0.9612/0.9520 for PLS-LDA; median class counts fell from 61 to 32 and from 88.5 to 71. On MetRef, PLS-LDA likewise improved its median CV accuracy from 0.9504 to 0.9924 and reduced median classes from 34 to 24, whereas KNN changed little and remained over-coarsened. Tcycle = 100 is therefore retained as a conservative search-depth setting, not as a claim of universal optimality; the MetRef PLS-LDA selected-run ARI was highest at Tcycle = 50.

The M sweep has a different interpretation. Increasing M raises the number of independent solutions entering the final agreement graph; it is not a search-depth parameter and is not expected to improve the externally scored best run monotonically. Each edge weight is the fraction of runs assigning its two samples to the same label. Its Bernoulli worst-case Monte Carlo standard error is 1/(2 sqrt(M)): 0.1118 at M = 20, 0.0707 at M = 50, and 0.0500 at M = 100. Empirically, the graph-weight RMSE relative to the M = 100 ensemble fell between M = 20 and M = 50 for every dataset/classifier pair, reaching 0.0180-0.0407 at M = 50 with correlations of 0.9888-0.9988. M = 100 is retained to bound ensemble uncertainty, while the nonmonotone selected-run ARI is reported rather than used to tune M.

Table 23: Tcycle sensitivity at fixed M = 100. Accuracy entries show Tcycle = 20 to Tcycle = 100.

| Dataset | Classifier | Best acc | Median acc | Median classes | Seconds |
|---|---|---|---|---|---|
| MetRef | KNN | 0.9924 -> 1.0000 | 0.8847 -> 0.8809 | 7 -> 9 | 3.116 -> 3.115 |
| MetRef | PLS-LDA | 0.9786 -> 1.0000 | 0.9504 -> 0.9924 | 34 -> 24 | 119.790 -> 584.089 |
| USPS | KNN | 0.9662 -> 0.9997 | 0.9414 -> 0.9925 | 61 -> 32 | 15.336 -> 15.828 |
| USPS | PLS-LDA | 0.9383 -> 0.9612 | 0.9220 -> 0.9520 | 88.5 -> 71 | 139.135 -> 636.650 |

Table 24: M sensitivity at fixed Tcycle = 100. Entries show M = 20 / 50 / 100.

| Dataset | Classifier | Best acc | Median acc | Selected ARI | Seconds |
|---|---|---|---|---|---|
| MetRef | KNN | 1.0000 / 1.0000 / 1.0000 | 0.9229 / 0.8931 / 0.8809 | 0.0517 / 0.0517 / 0.0517 | 0.464 / 1.235 / 3.115 |
| MetRef | PLS-LDA | 0.9985 / 1.0000 / 1.0000 | 0.9901 / 0.9939 / 0.9924 | 0.8257 / 0.8439 / 0.7700 | 113.381 / 277.456 / 584.089 |
| USPS | KNN | 0.9976 / 0.9976 / 0.9997 | 0.9947 / 0.9925 / 0.9925 | 0.3370 / 0.3370 / 0.3139 | 3.431 / 8.227 / 15.828 |
| USPS | PLS-LDA | 0.9639 / 0.9602 / 0.9612 | 0.9534 / 0.9503 / 0.9520 | 0.1098 / 0.1080 / 0.1156 | 123.851 / 319.838 / 636.650 |

Table 25: Convergence of agreement-graph edge weights. RMSE entries show M = 10 / 20 / 50 relative to M = 100.

| Dataset | Classifier | RMSE to M100 | Correlation at M50 |
|---|---|---|---|
| MetRef | KNN | 0.1377 / 0.0783 / 0.0407 | 0.9888 |
| MetRef | PLS-LDA | 0.0588 / 0.0453 / 0.0180 | 0.9988 |
| USPS | KNN | 0.1011 / 0.0617 / 0.0319 | 0.9955 |
| USPS | PLS-LDA | 0.1191 / 0.0803 / 0.0380 | 0.9926 |

## 6 Availability and reproducibility

kodama-cpp is intended for release in the public tkcaccia/kodama-cpp GitHub repository. Original project code is licensed under MIT, while identified compatible third-party portions

Table 26: Implementation claims and validation evidence.

| Feature claim | Where implemented | Test proving it | Benchmark proving benefit |
|---|---|---|---|
| Float32 numerical paths | MatrixView float overload in include/kodama/kodama.hpp; DenseF PLS-LDA workspaces in src/plscv.cpp; CUDA and Metal device buffers remain float32. | tests/test_cv.cpp and tests/test_metal.cpp check float32 KNNCV/PLSLDACV outputs, backend metadata, requested components, and accuracy thresholds. | The benchmark suite loads contiguous float32 matrices and records CPU, CUDA, and Metal runtime and accuracy separately. |
| Package-owned CPU HNSW | src/native_knn.cpp implements float32 HNSW search used by KNNCV, CoreKNN, graph construction, and KODAMA.matrix without linking FAISS. | Dependency-free CPU CTest covers folds, prediction accuracy, graph construction, and KODAMA optimization; no FAISS build option remains. | On MetRef, native CPU KNNCV reproduced accuracy 0.827033; runtime evidence is reported independently from accelerator paths. |
| CUDA nearest-neighbor search | src/native_cuda_backend.cu implements package-owned float32 exact KNN, signed-hash IVF-Flat, GPU k-means, resident inverted lists, exact-pilot recall tuning, and bounded row-batched exact k-means assignment; KNNCV_CUDA and CoreKNN call it directly. | CUDA tests explicitly exercise exact and IVF index types, tuning metadata, constrained folds, float32 CoreKNN, and end-to-end KODAMAMatrix_CUDA initialization. A clean build and ldd audit exclude FAISS/cuVS/RAFT/RMM; the 44,808-sample release run activates the bounded assignment path across four lanes. | Native IVF spot checks produced 0.973857 in 4.233 s on MNIST70k and 0.816724 in 0.195 s on MetRef. Macosko2015 retina completed M=100/Tcycle=100 in 97.295 s for KNN and 708.579 s for PLS-LDA on a 16 GiB GPU. |
| Label-aware SIMPLS PLS-LDA | src/plscv.cpp uses label/class inputs in CPU, CUDA, and Metal float32 SIMPLS paths; accelerator implementations avoid dense one-hot responses where possible. | CPU/CUDA/Metal tests check PLSLDACV sizes, constrained folds, selected component reporting, backend metadata, and accuracy thresholds. | CUDA PLSLDACV rows report 27.2x speedup on MetRef; Metal reproduced 50-component MetRef accuracy exactly with a 3.2x speedup over the dependency-free CPU build. |

Table 27: Implementation claims and validation evidence (continued).

| Feature claim | Where implemented | Test proving it | Benchmark proving benefit |
|---|---|---|---|
| Reusable fold and data buffers | PLSFoldXCacheF in src/plscv.cpp caches fold assignments, train/validation indices, scaled matrices, and CUDA Gram buffers; CoreKNN precomputes fold neighbors. | tests/test_cv.cpp checks CoreKNN/CorePLSLDA result sizes, worker/scheduler metadata, and that optimization does not decrease initial CV accuracy. | CorePLSLDA rows report 26.5x speedup on MetRef and 10.4x speedup on USPS for the CUDA core path. |
| Native Apple Metal backend | src/metal_backend.mm owns persistent Metal state, exact and IVF-Flat KNN, Lloyd k-means, MPS matrix products, label-aware SIMPLS, and latent-space LDA. | tests/test_metal.cpp exercises exact/IVF KNNCV, graphs, CoreKNN/CorePLSLDA, matrix KODAMA, graph-input KODAMA, and explicit rejection of unavailable Metal clustering. | MetRef exact KNN and PLS-LDA matched CPU accuracy; MNIST10k recall-tuned IVF preserved exact accuracy while reducing KNNCV time from 2.054 to 1.662 s. |
| Graph-input KODAMA | The graph-input entry points in src/kodama_matrix.cpp accept supplied neighbor indices/distances; the KNN path reuses the graph and the PLS-LDA graph-only path builds self-tuning Laplacian features. | R and Python wrapper tests exercise matrix_graph with KNN and PLS-LDA, and C++ tests cover graph construction, clustering, and visualization outputs. | Graph-input is presented as an optional API. The manuscript separately states that graph-only PLS-LDA should not be interpreted as equivalent to data-input PLS-LDA without matched benchmark evidence. |
| Float32 randomized PCA | src/pca.cpp supplies package-owned float32 preprocessing, sketching, QR, compact eigensolve, orientation, and variance reporting; src/pca_cuda.cu and the Metal matrix-multiply adapter execute dominant products on the selected accelerator. | tests/test_cv.cpp checks dimensions, finite values, centered scores, orthonormal loadings, ordered singular values, determinism, float/double input parity, and CUDA agreement; tests/test_metal.cpp checks Metal agreement; R and Python tests exercise the public wrappers. | PCA is an auxiliary public primitive rather than part of the KODAMA objective. Performance claims will be reported separately from KODAMA.matrix timings when a matched PCA benchmark is archived. |

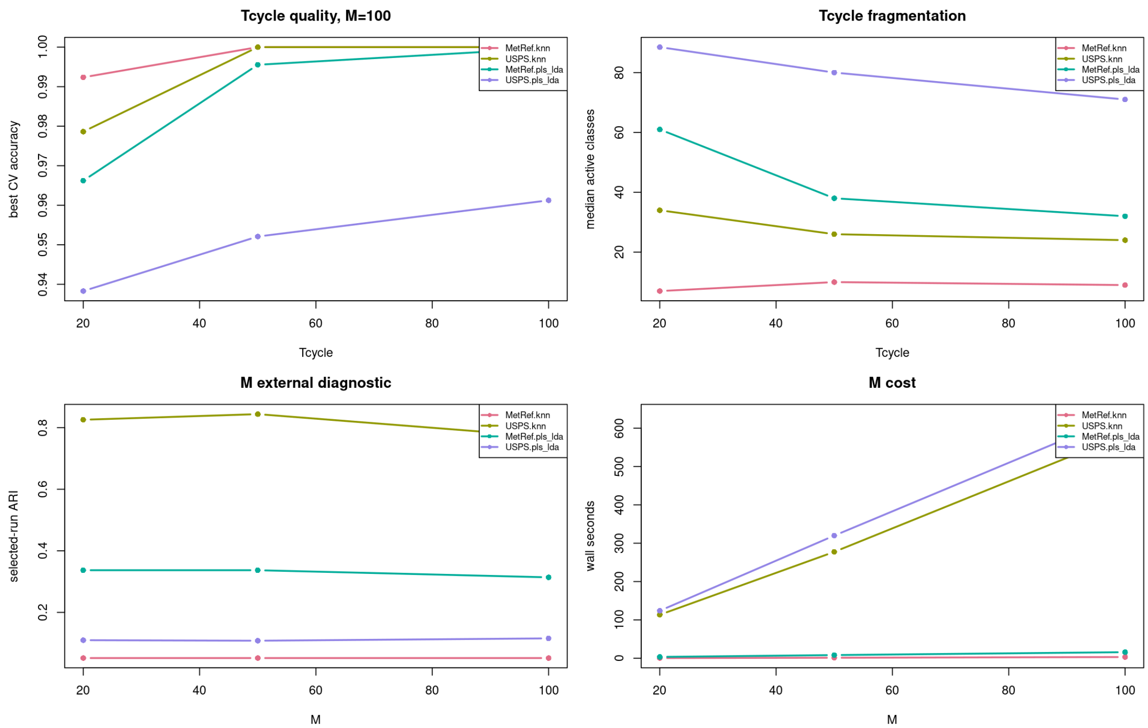


Figure 3: Sensitivity of KODAMA quality and runtime to Tcycle and M on named nonspatial datasets.

retain their upstream terms. In particular, the Metal backend is marked MIT AND Apache-2.0 because its fastEmbedR lineage retains the Faiss-mlx fused list-scan/top-k organization under Apache-2.0. The recommended project split is a standalone C++17 core repository, an R wrapper repository, and a Python wrapper repository.

Reproducibility is supported through CMake builds, CPU, CUDA, and Metal tests, wrapper validation tests, benchmark scripts, and explicit recording of backend parameters. GitHub Actions builds and tests the CPU core on Linux and macOS, builds the native Metal core on macOS, and installs and tests both wrapper packages. A separate LLVM workflow measures CPU source coverage, while Doxygen publishes the typed API and a compact C++/R/Python walkthrough at https://tkcaccia.github.io/kodama-cpp/. The C++ core does not depend on R data readers; wrappers translate host-language objects into contiguous matrices before calling the library.

A versioned provenance audit maps every shipped source-like file to an SPDX license identifier and records pinned KODAMA, fastPLS, fastEmbedR, faissR, FAISS, Faiss-mlx, and cuVS snapshots. Stefano Cacciatore confirmed that tkcaccia is his historical Git identity and separately authorized MIT distribution of the KODAMA and fastPLS-derived portions whose copyright he owns. The audit preserves publication coauthor credit, third-party notices, and full license texts, and is enforced by a release-time source-header and retained-license checksum test.

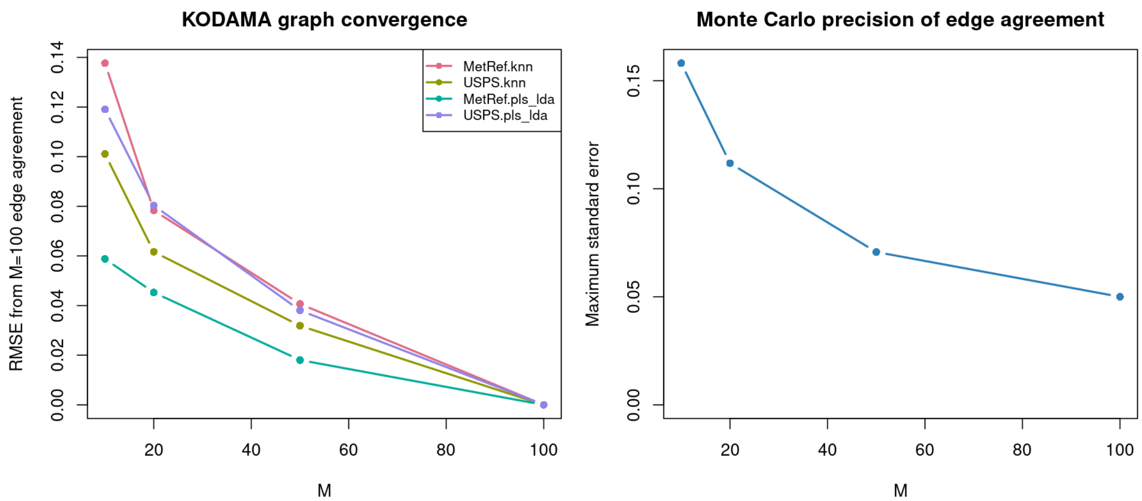


Figure 4: Convergence of edge-agreement weights as independent KODAMA runs are added. RMSE is measured against M = 100; the right panel shows the Bernoulli worst-case Monte Carlo standard error.

### 6.1 Installation paths

### 6.2 Wrapper validation

### 6.3 License and dependency notes

## 7 Limitations

The current implementation prioritizes KNN and PLS-LDA because they are central to the KODAMA optimization principle and can be implemented cleanly on CPU, CUDA, and Apple Metal. Additional classifiers are outside this release unless they preserve the cross-validated accuracy objective and can be tested without changing the method for a specific benchmark.

The main engineering limitation is that KODAMA performs many cross-validation fits. This makes memory locality, buffer reuse, label compaction, and GPU scheduling as important as asymptotic complexity. The present release includes float32 buffers, fold reuse, label compaction, CUDA kernels, and native Metal KNN and PLS-LDA kernels, but full accelerator-resident batching of independent M runs remains an engineering extension rather than a claim of the current manuscript.

The native CUDA neighbor-graph builder currently supports at most 256 retained neighbors per sample, whereas the CPU path supports larger rows. The implementation raises an error rather than silently truncating k. This matters when comparing with interfaces that couple landmark count and graph size; the current-CRAN MetRef systems comparison therefore reports the exactly matched CPU4 path and does not manufacture an unmatched CUDA row.

Table 28: Release-validation evidence tracked by the project.

| Area | Current evidence |
|---|---|
| Correctness | CTest passed on dependency-free CPU, Apple Metal, and CUDA builds; tests cover constrained folds, predictions, confusion matrices, requested component counts, and backend parity. |
| Numerics | Float32 MatrixView validation tests exercise CPU, CUDA, and Metal KNN/PLS-LDA paths with typed backend metadata; a fresh CUDA configure/build verifies that SIMPLS has no Armadillo or double-fallback dependency. |
| Performance | Kernel-level KNNCV/PLSLDACV timings are separated from core optimizer and KODAMA.matrix timings in the benchmark outputs. |
| Continuous integration | GitHub Actions commit 0e019c1 passed CPU core jobs on Linux and macOS, a native Metal job on macOS, R package build/check, and Python installation/pytest. |
| Coverage and documentation | LLVM 22.1.1 CPU instrumentation measured 63.58% line, 57.03% branch, 66.04% function, and 67.61% region coverage. CUDA and Metal are excluded from that CPU number and retain hardware tests. Doxygen generation and GitHub Pages deployment passed, and the public API site is reachable at https://tkcaccia.github.io/kodama-cpp/. |
| Release | CMake install targets, wrapper build scripts, benchmark drivers, SPDX source headers, a pinned provenance matrix, retained third-party license texts, and a checksum-backed license audit are present. The final clean tag, commit hash, archive digest, and DOI are required by RELEASE_CHECKLIST.md and will be inserted only after they are created. |

## 8 KODAMA pseudocode

Algorithm 1 isolates one independent run inside the M-run ensemble. It starts from one landmark label vector, uses the previous CV predictions to propose label changes, evaluates each proposal with exactly one CV pass, and returns the best label vector found by that run.

```
Algorithm 1. One independent KODAMA run

Input:
  X_L: landmark matrix for this run
  y0: initial label vector on landmarks
  groups: movable units, either samples or constrained groups
  fixed: optional fixed-label mask
  folds: CV fold assignment
  F: classifier in {KNN, PLS-LDA}
  Tcycle: proposal/evaluation cycles

Output:
  y_best: optimized labels for this run
  pred_best: CV predictions attached to y_best
  acc_best: raw CV accuracy of y_best

pred0 <- CrossValidate(F, X_L, y0, folds)
acc0 <- Accuracy(y0, pred0)
score0 <- ObjectiveScore(y0, acc0)
y_best <- y0; pred_best <- pred0; acc_best <- acc0; score_best <- score0
y_current <- y0; pred_current <- pred0; acc_current <- acc0; score_current <- score0
```

Table 29: Installation and runtime checks for the standalone core and wrappers.

| Target | Command or check |
|---|---|
| C++ CPU | Configure with CMake using KODAMA_ENABLE_CUDA=OFF and KODAMA_ENABLE_METAL=OFF; then build, test, and install the exported target. |
| C++ Metal | On macOS, configure with KODAMA_ENABLE_METAL=ON while CUDA is off; then build and run CTest. Apple system frameworks are linked automatically. |
| C++ CUDA | Activate an environment containing the CUDA toolkit; configure with KODAMA_ENABLE_CUDA=ON; then build and run CTest. FAISS, cuVS, RAFT, RMM, Armadillo, and external graph-clustering libraries are not required. |
| R wrapper | Set KODAMA_CPP_ROOT and KODAMA_CPP_BUILD_DIR, then run R CMD INSTALL kodama-r. The wrapper links the installed or selected core build. |
| Python wrapper | Pass KODAMA_CPP_ROOT and KODAMA_CPP_BUILD_DIR as CMake config settings to pip when installing kodama-python. |
| Runtime check | Run ctest for the core library; in R call KODAMA.diagnostics(); in Python import kodama and run the package validation tests. |

```
for t = 1,...,Tcycle do
  if evolutionary_search then
    y_base <- y_current; pred_base <- pred_current
  else
    y_base <- y_best; pred_base <- pred_best
  end if

  y_prop <- y_base
  qmax <- ProposalBudget(t, Tcycle, number_of_groups)
  q <- UniformInteger(1, qmax)
  selected_groups <- SampleWithoutReplacement(groups, q)

  for each group g in selected_groups do
    E <- {i in members(g): fixed[i] != 1}
    if E is empty then continue
    replacement <- SampleEmpirical(pred_base[E])
    y_prop[E] <- replacement
  end for

  y_prop <- ClassTransitionProposals(y_prop, pred_base, fixed)
  pred_prop <- CrossValidate(F, X_L, y_prop, folds)  # exactly one CV pass
  acc_prop <- Accuracy(y_prop, pred_prop)
  score_prop <- ObjectiveScore(y_prop, acc_prop)

  if score_prop > score_best then
    y_best <- y_prop; pred_best <- pred_prop
    acc_best <- acc_prop; score_best <- score_prop
  end if

  if evolutionary_search and
     Accept(score_prop, score_current, acc_current, t, Tcycle, rng) then
    y_current <- y_prop; pred_current <- pred_prop
    acc_current <- acc_prop; score_current <- score_prop
  end if
```

Table 30: Local CPU/Metal and CUDA validation results for the core and wrapper packages.

| Target | Validation result |
|---|---|
| C++ core, local CPU | Dependency-free release-candidate build passed 4/4 configured tests on macOS, including the source/license audit and frozen public-API snapshot. |
| C++ core, Apple Metal | Native Metal release-candidate build passed 5/5 configured tests, including the source/license audit and frozen public-API snapshot; a clean external CMake consumer linked the installed package and selected Metal. |
| C++ core, CUDA | A fresh CUDA 13.2 release-candidate build without FAISS, cuVS, RAFT, RMM, or Armadillo succeeded on chiamaka and CTest passed 4/4, including the source/license audit, core tests, frozen public-API test, and float32 install-consumer smoke test. The linked-soname audit found none of the removed dependencies, and the CMake cache contained no corresponding package option, target, header, or library entry; only the CUDA Toolkit environment directory retained a legacy faissgpu-cuvs label. |
| R wrapper, local CPU | R CMD build followed by R CMD check –as-cran –no-manual passed on the source tarball in a UTF-8 locale with only the expected new-submission NOTE. |
| Python wrapper, local CPU | Temporary virtual-environment install against the local CPU build passed pytest: 4/4 tests. |
| GitHub Actions | Commit 0e019c1 passed five CI jobs: CPU core on Ubuntu and macOS, native Metal on macOS, R package build/check on Ubuntu, and Python installation/pytest on Ubuntu. Separate coverage and API-documentation workflows also passed. |

```
  if acc_prop == 1 then break
end for

return y_best, pred_best, acc_best
```

### 8.1 KODAMA.matrix ensemble

Algorithm 2 repeats Algorithm 1 independently M times, projects each optimized run back to all samples, and builds the final KODAMA-corrected graph from agreement across the optimized label vectors.

```
Algorithm 2. KODAMA.matrix ensemble

Input:
  X: n by p data matrix
  F: classifier in {KNN, PLS-LDA}
  M: number of independent runs
  Tcycle: proposal/evaluation cycles per run
  landmarks, splitting, graph_neighbors
  optional starting_labels, constrain, fixed

Output:
```

Table 31: Release notes for core components and external dependencies.

| Component | Release note |
|---|---|
| kodama-cpp project code | MIT, with SPDX copyright assigned to Stefano Cacciatore; historical Git identity tkcaccia. |
| KODAMA-derived code | Public origin GPL (>= 2); local portions whose copyright Stefano Cacciatore owns are separately authorized under MIT. KODAMA coauthors remain credited. |
| fastPLS-derived code | Public origin GPL-3; local SIMPLS/LDA portions whose copyright Stefano Cacciatore owns are separately authorized under MIT. fastPLS coauthors remain credited. |
| fastEmbedR and faissR | MIT snapshots pinned for randomized-PCA policy, UMAP/openTSNE, HNSW, Metal, and exact 2D/3D grid-KNN lineage. |
| Native CPU HNSW | Direct FAISS-derived organization under MIT; Meta and Stefano Cacciatore notices and the complete FAISS license are retained. |
| Apple Metal | MIT AND Apache-2.0 for the combined file; Meta, Faiss-mlx, and modification notices plus both full licenses are retained. |
| Native CUDA | Package-owned exact/IVF KNN, k-means, and SIMPLS/LDA use CUDA Toolkit libraries without FAISS, cuVS, RAFT, RMM, Armadillo, or external graph-clustering links. |
| Audit enforcement | All tracked source-like files carry SPDX identifiers; official license texts are SHA-256 checked by tools/check_license_headers.sh. |

```
  C: M by n matrix of optimized label vectors
  A: length-M vector of best raw CV accuracies
  G_K: KODAMA-corrected neighbor graph

X32 <- float32(X)
G0 <- KNNGraph(X32, graph_neighbors)
for r = 1,...,M do
  rng <- RNG(seed + r)
  L <- SelectLandmarks(X32, landmarks, rng)
  groups <- ConstrainedGroups(constrain[L])
            or one singleton group per landmark
  y0 <- InitialLabels(X32[L], starting_labels[L], splitting, groups, rng)
  folds <- AssignCVFolds(groups, y0)

  y_best, pred_best, acc_best <- OneIndependentRun(
      X32[L], y0, groups, fixed[L], folds, F, Tcycle, rng)

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X32, L, F, groups, G0)
  A[r] <- acc_best
end for

G_K <- ReweightGraphByLabelAgreement(G0, C)
return C, A, G_K

ObjectiveScore(y, a):
  score <- a
  if guarded_diversity then
    p_l <- class proportions in y
    score <- score * sqrt(1 - sum_l p_l^2)
  end if
  if class_coarsening then
    score <- score - (1 - a) * Parsimony(y)
```

```
  end if
  return score

Accept(s_new, s_old, a_old, t, Tcycle, rng):
  if s_new >= s_old then return TRUE
  tau <- max(1e-9, 0.10 * max(0, 1 - a_old) * (1 - t / Tcycle))
  return Uniform(0, 1, rng) < exp((s_new - s_old) / tau)
```

### 8.2 KODAMA.matrix.graph from supplied neighbors

Algorithm 3 keeps the same label-evolution objective while replacing the initial neighbor search with caller-supplied indices and distances. KNN uses the supplied graph directly; PLS-LDA uses self-tuning Laplacian features only when the original feature matrix is not supplied.

```
Algorithm 3. KODAMA.matrix.graph from supplied neighbors

Input:
  I, D: n by k neighbor indices and distances
  optional X: original n by p data matrix
  F: classifier in {KNN, PLS-LDA}
  M, Tcycle, landmarks, splitting
  optional starting_labels, constrain, fixed

Output:
  C: M by n matrix of optimized label vectors
  A: length-M vector of best raw CV accuracies
  G_K: KODAMA-corrected graph on the supplied neighborhoods

G0 <- NormalizeExternalGraph(I, D)
if X is supplied then
  X_work <- float32(X)
else if F == PLS-LDA then
  X_work <- SelfTuningLaplacianFeatures(G0, ncomp)
else
  X_work <- empty
end if

for r = 1,...,M do
  rng <- RNG(seed + r)
  L <- SelectLandmarksFromGraphOrData(G0, X_work, landmarks, rng)
  groups <- ConstrainedGroups(constrain[L])
            or one singleton group per landmark
  y0 <- InitialLabelsFromGraphOrData(G0[L], X_work[L], starting_labels[L],
                                     splitting, groups, rng)
  folds <- AssignCVFolds(groups, y0)

  if F == KNN then
    y_best, pred_best, acc_best <- Algorithm1(G0[L], y0, groups, fixed[L], folds, KNN, Tcycle)
  else
    y_best, pred_best, acc_best <- Algorithm1(X_work[L], y0, groups, fixed[L], folds, PLS-LDA, Tcycle)
  end if

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X_work, L, F, groups, G0)
  A[r] <- acc_best
end for

G_K <- ReweightGraphByLabelAgreement(G0, C)
return C, A, G_K
```

## Acknowledgments and Disclosure of Funding

The authors thank contributors to the KODAMA, fastPLS, and fastEmbedR software projects.

## References

Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.

Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.

Stefano Cacciatore and Leonardo Tenori. *KODAMA: Knowledge Discovery by Accuracy Maximization*, 2026. R package documentation, CRAN.

Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.

Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.

Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. MIT Press, 2006.

Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.

Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2: 193–218, 1985.

Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.

Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995.

Nikolaus Kriegeskorte, W. Kyle Simmons, Patrick S. F. Bellgowan, and Chris I. Baker. Circular analysis in systems neuroscience: the dangers of double dipping. *Nature Neuroscience*, 12:535–540, 2009.

Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.

George C. Linderman, Manas Rachh, Jeremy G. Hoskins, Stefan Steinerberger, and Yuval Kluger. Fast interpolation-based t-sne for improved visualization of single-cell rna-seq data. *Nature Methods*, 16:243–245, 2019.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

Alexander Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. Data programming: Creating large training sets, quickly. In *Advances in Neural Information Processing Systems*, 2016.

Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

Sören Sonnenburg, Mikio L. Braun, Cheng Soon Ong, Samy Bengio, Leon Bottou, Geoffrey Holmes, Yann LeCun, Klaus-Robert Müller, Fernando Pereira, Carl Edward Rasmussen, Gunnar Rätsch, Bernhard Schölkopf, Alexander Smola, Pascal Vincent, Jason Weston, and Robert C. Williamson. The need for open source software in machine learning. *Journal of Machine Learning Research*, 8:2443–2466, 2007.

Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.

Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006.

Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems*, 2003.

Xiaojin Zhu, Zoubin Ghahramani, and John Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 912–919, 2003.